

◇ PROJECT DEEP-DIVE

Carbon Credit Tokenization Protocol

IoT-verified carbon offsets, minted as on-chain tokens, tradeable with full provenance and permanent retirement tracking.

5

SMART CONTRACTS

3

EXTERNAL ORACLES

~36h

BUILD TIMELINE

\$420B

TARGET MARKET

Solidity

ERC-1155

Chainlink

IPFS

Uniswap v3

IoT Attestation

Next.js

NAVIGATION

Table of Contents

01	Executive Summary — The problem, solution, and why now	3
02	Problem Space — Why voluntary carbon markets are broken	3
03	Solution Architecture — Full system overview and component map	4
04	Smart Contract Design — All 5 contracts, interfaces, and state machines	5
05	Data Flow & Token Lifecycle — From IoT sensor to burned credit	6
06	Tech Stack Breakdown — Every tool, library, and service	7
07	Frontend Architecture — UI flows, components, and Web3 hooks	8
08	Agent Build Prompts — Copy-paste prompts for Claude/Cursor/Copilot	9
09	Vibe Coder Tips — Speed-running the build, gotchas, shortcuts	11
10	Build Timeline — Hour-by-hour hackathon schedule	12
11	Risk & Mitigation — What will go wrong and how to handle it	12
12	Demo Script — The exact 3-minute pitch judges will remember	13
13	Post-Hackathon Roadmap — V1 → V2 → mainnet path	14

01 — OVERVIEW

Executive Summary

What it is, why it matters, and why it can only exist on-chain

The Carbon Credit Tokenization Protocol bridges the physical world of carbon sequestration with the financial world of on-chain markets. Real IoT sensors on reforestation and soil carbon projects attest sequestration data via Chainlink oracles. Each verified tonne of CO₂ removed mints an ERC-1155 token with full, immutable provenance. Credits trade on a Uniswap v3 pool and retire permanently via a burn function — replacing an industry notorious for double-counting and fraud with cryptographic certainty.

\$2

AVG PRICE PER
TONNE, LEGACY
MARKET

**89%**

OF OFFSETS NEVER
INDEPENDENTLY
VERIFIED

**3×**

DOUBLE-COUNTING
RATE, VOLUNTARY
MARKET

**\$420B**

PROJECTED MARKET
SIZE BY 2030



Why This Can Only Be a Blockchain Project

**Immutable
Provenance**

Every credit's birth, ownership chain, and retirement is permanently recorded. No central party can alter history.

Trustless Settlement

Buyers and sellers transact without brokers. Smart contracts enforce rules without counterparty risk.

**Permissionless
Access**

Small reforestation projects in developing nations can list credits globally without intermediaries extracting 40–60%.

Verifiable Retirement

Token burns are public and permanent. No credit can be sold twice. The retirement event is the receipt.

02 — PROBLEM SPACE

Why Voluntary Carbon Markets Are Broken

The Core Failure Modes

Problem	Current Reality	Our Solution
Double Counting	Same offset sold to multiple buyers across registries. No shared ledger.	On-chain uniqueness. Each token ID is globally unique. Burns are final.
Stale Audits	One-time audit from 3 years ago represents "current" sequestration state.	Real-time IoT attestation. Chainlink updates sequestration readings continuously.
Opaque Retirement	Retirement certificates are PDFs from private registries. Unverifiable.	ERC-1155 burn() emits permanent on-chain RetirementEvent with buyer identity and timestamp.
High Intermediary Costs	Brokers extract 40–60% margin between project and buyer.	Uniswap v3 pool. 0.3% fee. No broker. Project receives 99.7% of trade value.
No Price Discovery	Prices negotiated OTC in spreadsheets. No public market.	AMM provides continuous public price discovery. Anyone can query the TWAP.

The Hackathon Pitch in One Sentence

"We replace a \$420B market built on PDF certificates and broker phone calls with IoT sensor data, ZK-attested proofs, and a burn function."

03 — ARCHITECTURE

Solution Architecture

Full system overview — from physical sensor to on-chain token



Component Dependency Map

Component	Depends On	Exposes To	Critical Path
IoT Sensors	Physical hardware	Chainlink node	Yes
Chainlink Oracle	IoT data feed, IPFS	CarbonToken contract	Yes
ProjectRegistry	Deployer wallet	CarbonToken, frontend	Yes
CarbonToken (ERC-1155)	ProjectRegistry, Chainlink	Uniswap pool, RetirementLedger	Yes
RetirementLedger	CarbonToken	Frontend, explorer	No (demo stub OK)
Uniswap v3 Pool	CarbonToken, USDC	Frontend swap UI	Partial (mock OK)
IPFS	Pinata API	CarbonToken metadata URI	Partial
Next.js Frontend	wagmi, viem, The Graph	End users	Yes

Hackathon MVP Scope

For a 36-hour build, implement: ProjectRegistry + CarbonToken + mock Chainlink feed + RetirementLedger + basic Next.js UI. Skip Uniswap integration — show price as a static display. The core value proposition (mint → trade → burn) must work end-to-end. Everything else is polish.

04 — SMART CONTRACTS

Smart Contract Design

5 contracts, their interfaces, storage layouts, and state transitions

Contract Overview

Contract	Standard	Key Functions	Deployer
ProjectRegistry.sol	Ownable	registerProject, approveProject, getProject	Protocol admin
CarbonToken.sol	ERC-1155	mintBatch, retire, uri, totalSupply	ProjectRegistry
OracleConsumer.sol	Chainlink Client	requestSequestration, fulfill, getLatestReading	CarbonToken
RetirementLedger.sol	Custom	recordRetirement, getByAddress, getByProject	CarbonToken
CarbonPoolFactory.sol	Uniswap v3 fork	createPool, addLiquidity, swap	Protocol admin

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "../ProjectRegistry.sol";
import "../RetirementLedger.sol";

contract CarbonToken is ERC1155, Ownable {
    // tokenId = projectId (unique per project)
    struct CreditMetadata {
        uint256 projectId;
        uint256 vintage;           // Year of sequestration
        uint256 tonnesCO2;        // Verified CO2 in kg (×1000)
        bytes32 iotDataHash;      // IPFS hash of raw sensor data
        uint256 oracleTimestamp;  // When oracle confirmed reading
        bool retired;
    }

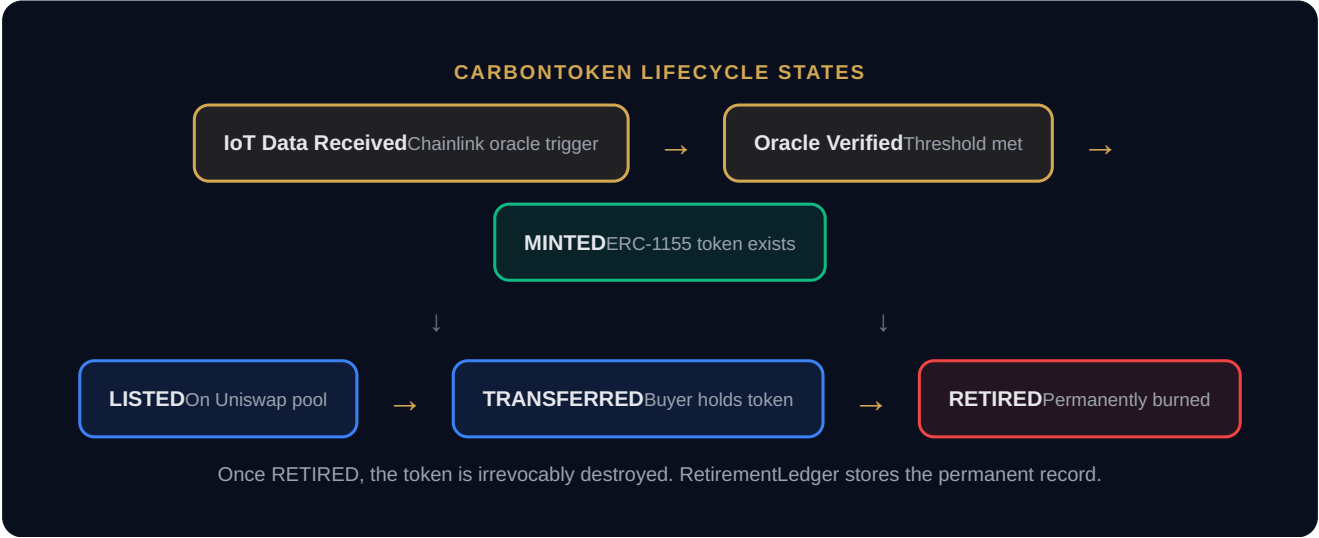
    mapping(uint256 => CreditMetadata) public credits;
    mapping(uint256 => uint256) public totalSupply;
    RetirementLedger public retirementLedger;

    event CreditMinted(uint256 indexed tokenId, uint256 amount, bytes32 iotDataHash);
    event CreditRetired(uint256 indexed tokenId, address indexed retiree, uint256 amount, string
reason);

    function mintCredit(
        uint256 projectId,
        uint256 vintage,
        uint256 tonnesCO2,
        bytes32 iotDataHash,
        uint256 amount
    ) external onlyOracle {
        uint256 tokenId = _generateTokenId(projectId, vintage);
        credits[tokenId] = CreditMetadata(projectId, vintage, tonnesCO2,
            iotDataHash, block.timestamp, false);

        totalSupply[tokenId] += amount;
        _mint(msg.sender, tokenId, amount, "");
        emit CreditMinted(tokenId, amount, iotDataHash);
    }

    function retire(uint256 tokenId, uint256 amount, string calldata reason) external {
        require(balanceOf(msg.sender, tokenId) >= amount, "Insufficient balance");
        _burn(msg.sender, tokenId, amount);
        retirementLedger.recordRetirement(msg.sender, tokenId, amount, reason);
        emit CreditRetired(tokenId, msg.sender, amount, reason);
    }
}
```

05 — DATA FLOW

Data Flow & Token Lifecycle

Every step from physical CO₂ removal to permanent on-chain retirement

End-to-End Transaction Flow

1 IoT Sensor Reads CO₂ Sequestration

ESP32 or industrial sensor reads soil carbon / biomass growth every 6 hours. Data signed with device private key. Raw reading: `{"co2_kg": 4200, "lat": -3.4, "lng": 37.2, "ts": 1719000000, "sig": "0x..."}`

2 Data Uploaded to IPFS + Chainlink Node

Sensor gateway uploads signed JSON to IPFS via Pinata. Returns CID. Simultaneously calls Chainlink Any API job, passing CID. Oracle fetches, verifies device signature, aggregates multi-sensor readings.

3 Oracle Threshold Check

OracleConsumer.sol receives callback. Checks: reading ≥ 1 tonne CO₂, device is registered for this projectId, data is fresh (timestamp within 7 days). If all pass → calls CarbonToken.mintCredit().

4 ERC-1155 Token Minted

mintCredit() creates tokenId = keccak256(projectId, vintage). Stores CreditMetadata on-chain. Mints N tokens (1 token = 1 tonne CO₂). Sets URI pointing to IPFS metadata JSON with sensor data, project info, audit report hash.

5 Token Listed on Uniswap v3 Pool

Project owner calls PoolFactory to create CCT/USDC pool or adds liquidity to existing pool. Price discovery begins. Buyers can swap USDC → CCT at the market rate. The Graph indexes all swaps and liquidity events.

6 Buyer Purchases Credits

Corporate buyer swaps USDC for CCT tokens through the frontend. wagmi handles wallet connection. Transaction confirms in ~2 seconds on Base. Buyer now holds verified, provenance-rich carbon credits in their wallet.

7 Credits Retired (Burned Permanently)

Buyer calls retire(tokenId, amount, "Q4 2024 Scope 3 offset"). ERC-1155 burn() executes. CreditRetired event emitted with: retiree address, tokenId, amount, reason, block timestamp. RetirementLedger.recordRetirement() stores full record. Retirement is now permanently verifiable by anyone.

Oracle Data Integrity Checks

Check	Method	Failure Action
Device signature valid	ECDSA recover against registered device pubkey	Revert, emit SuspiciousData event
Timestamp freshness	block.timestamp - reading.ts < 7 days	Revert with StaleData error
Minimum threshold	tonnesCO2 $\geq$ MIN_TONNES (default: 1000 kg)	Queue for next cycle
Project is active	ProjectRegistry.getProject(projectId).active == true	Revert with InactiveProject
No duplicate minting	credits[tokenId].oracleTimestamp == 0	Revert with AlreadyMinted
Multi-oracle consensus	≥ 3 of 5 oracle nodes agree within 5%	Request new reading after 1hr

06 — TECH STACK

Tech Stack Breakdown

Every tool, library, and external service with version recommendations

Smart Contract Layer

Tool	Version	Purpose
solidity	^0.8.20	Smart contract language
foundry	latest	Testing + deployment
hardhat	2.22	Local devnet + scripts
openzeppelin	5.x	ERC-1155, Ownable, etc.
chainlink/contracts	0.8.0	Oracle consumer base
@uniswap/v3-core	1.0.1	Pool creation
@uniswap/v3-periphery	1.4.4	Swap router

Frontend Layer

Tool	Version	Purpose
next.js	14.x	React framework
wagmi	2.x	Web3 React hooks
viem	2.x	Ethereum interactions
@tanstack/query	5.x	Data fetching
tailwindcss	3.x	Styling
recharts	2.x	Price/sequestration charts
framer-motion	11.x	Animations

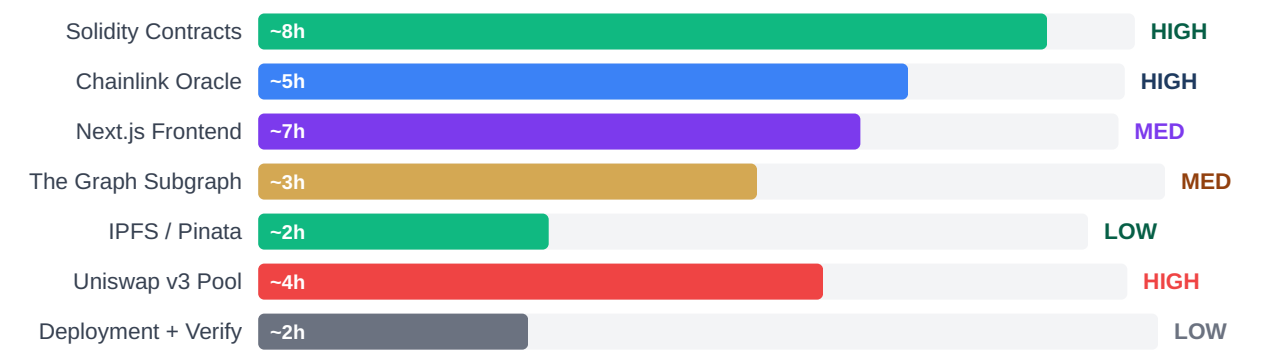
Infrastructure & Services

Service	Purpose	Free Tier?
Base Sepolia	Testnet deployment	Yes
Chainlink Any API	IoT oracle	Yes (testnet)
Pinata	IPFS pinning	Yes (1GB)
The Graph	Event indexing	Yes
Alchemy	RPC provider	Yes (300M/mo)
Vercel	Frontend hosting	Yes

Dev Tooling

Tool	Purpose
forge test	Contract unit tests
forge coverage	Coverage report
slither	Static analysis
etherscan-verify	Contract verification
graph-cli	Subgraph deploy
dotenv	Env management

Technology Complexity vs. Build Time



07 — FRONTEND

Frontend Architecture

Page structure, Web3 hooks, and key UI components

Application Routes

Route	Page	Key Components	Data Source
/	Landing / Dashboard	Stats banner, recent retirements, live price	The Graph, Uniswap SDK
/projects	Project Marketplace	Project cards, filter by type/region/vintage	ProjectRegistry contract
/projects/[id]	Project Detail	IoT chart, sensor map, buy button, audit docs	Contract + IPFS
/buy	Purchase Flow	Uniswap widget, token selector, amount input	Uniswap v3 SDK
/retire	Retirement Flow	Holdings list, retire form, certificate gen	CarbonToken contract
/certificate/[id]	Retirement Cert	PDF-style cert, on-chain verification badge	RetirementLedger
/dashboard	Portfolio	Holdings, history, offset progress bar	wagmi + The Graph
/onboard	Project Onboarding	Registry form, sensor config, IPFS upload	ProjectRegistry + Pinata

TYPESCRIPT

hooks/useCarbonToken.ts

```
import { useReadContract, useWriteContract, useWaitForTransactionReceipt } from 'wagmi';
import { CARBON_TOKEN_ABI, CARBON_TOKEN_ADDRESS } from '@contracts';

export function useCarbonToken() {
  // Read token balance
  const { data: balance } = useReadContract({
    abi: CARBON_TOKEN_ABI,
    address: CARBON_TOKEN_ADDRESS,
    functionName: 'balanceOf',
    args: [address, tokenId],
  });

  // Retire credits
  const { writeContract, data: hash } = useWriteContract();
  const { isLoading, isSuccess } = useWaitForTransactionReceipt({ hash });

  const retireCredits = (tokenId, amount, reason) => {
    writeContract({
      abi: CARBON_TOKEN_ABI,
      address: CARBON_TOKEN_ADDRESS,
      functionName: 'retire',
      args: [tokenId, amount, reason],
    });
  };

  return { balance, retireCredits, isLoading, isSuccess };
}
```

IoT Sequestration Chart Component

The Most Impressive UI Element

Build a real-time chart showing CO₂ sequestration readings over time, pulled from IPFS-stored sensor logs. Use Recharts AreaChart with gradient fill. Animate data coming in. This is what makes the demo visceral — judges see a live chart of carbon being removed from the atmosphere, with each data point verifiably tied to an on-chain token. Build this first, it's your "wow moment."

08 — AI AGENT PROMPTS

Agent Build Prompts

Copy-paste prompts for Claude, Cursor, and GitHub Copilot

How to Use These Prompts

Each prompt is designed to be self-contained. Give the agent the prompt + any existing code context. For Cursor: open the relevant file first. For Claude Code: run from the project root. For ChatGPT/Claude web: paste directly. Prompts are ordered by build sequence — do them in order.

Prompt 01 — Project Setup

Claude Code / Cursor

Initialize a new Web3 project called "carbon-credit-protocol" with this exact structure:

- /contracts (Foundry project with forge init)
- /frontend (Next.js 14 app router, TypeScript, Tailwind CSS)
- /scripts (deployment and oracle simulation scripts)
- /subgraph (The Graph subgraph for event indexing)

Install these dependencies:

Contracts: openzeppelin-contracts@5.x, chainlink/contracts@0.8.0, @uniswap/v3-core, @uniswap/v3-periphery

Frontend: wagmi@2, viem@2, @tanstack/react-query@5, recharts, framer-motion

Create a .env.example with all required env vars:

PRIVATE_KEY, ALCHEMY_API_KEY, CHAINLINK_TOKEN, CHAINLINK_ORACLE, PINATA_JWT, ETHERSCAN_API_KEY, NEXT_PUBLIC_ALCHEMY_RPC

Set up foundry.toml with remappings for all imports. Create a basic README with architecture overview.

Prompt 02 — Core Contracts

Claude Code / Cursor

Write 4 Solidity contracts for a carbon credit tokenization protocol:

1. ProjectRegistry.sol
 - registerProject(name, location, sensorIds[], ipfsDocHash) → projectId
 - approveProject(projectId) – onlyOwner
 - struct Project { name, location, sensorIds, docHash, active, registeredAt }
 - Events: ProjectRegistered, ProjectApproved
2. CarbonToken.sol (ERC-1155)
 - mintCredit(projectId, vintage, tonnesCO2, iotDataHash, amount) – onlyOracle modifier
 - retire(tokenId, amount, reason) – burns tokens, calls RetirementLedger
 - uri(tokenId) returns IPFS metadata URI
 - struct CreditMetadata { projectId, vintage, tonnesCO2, iotDataHash, oracleTimestamp, retired }
 - Events: CreditMinted, CreditRetired
3. OracleConsumer.sol (Chainlink Any API)
 - requestSequestration(projectId, ipfsCID) – calls Chainlink job
 - fulfill(requestId, co2Reading, dataHash) – callback, calls CarbonToken.mintCredit if threshold met
 - MIN_TONNES_THRESHOLD = 1000 (kg)
4. RetirementLedger.sol
 - recordRetirement(retiree, tokenId, amount, reason) – onlyToken
 - getRetirementsByAddress(address) returns RetirementRecord[]
 - getRetirementsByProject(projectId) returns RetirementRecord[]
 - struct RetirementRecord { retiree, tokenId, amount, reason, timestamp }
 - Events: RetirementRecorded

Use OpenZeppelin 5.x. Add NatSpec comments. Include custom errors. Target Solidity ^0.8.20.

Prompt 03 — Foundry Tests

Claude Code / Cursor

Write comprehensive Foundry tests for CarbonToken.sol and RetirementLedger.sol.

Test file: test/CarbonToken.t.sol

Cover these scenarios:

1. testMintCredit_success – oracle mints, metadata stored correctly, event emitted
2. testMintCredit_revertNotOracle – non-oracle cannot mint
3. testRetire_success – holder retires, tokens burned, ledger updated, event emitted
4. testRetire_revertInsufficientBalance – cannot retire more than held
5. testRetire_recordsInLedger – RetirementLedger.getRetirementsByAddress returns correct record
6. testTokenUri_returnsCorrectIPFSHash – URI format is ipfs://{iotDataHash}
7. testTotalSupply_decreasesOnRetirement – supply drops after burn
8. testBatchMint_multipleVintages – different vintages get different tokenIds

Use vm.prank() for access control tests. Use forge-std console.log for debug output.

Include a MockChainlinkOracle contract in test/mocks/. Run with: forge test -vvv

Build a Next.js 14 app router page at `app/projects/[id]/page.tsx` for a carbon credit project detail page.

Design: dark theme (`#0A0F1E` background), green (`#10B981`) accent color, Inter font.

Include these sections:

1. Header – project name, location flag, verification badge ("IoT Verified ✓"), live sequestration rate
2. IoT Sequestration Chart – Recharts AreaChart showing CO2 removal over last 30 days. Data from IPFS JSON (mock with realistic data for now). Green gradient fill. Animated on mount with framer-motion.
3. Credit Stats row – 4 metric cards: Total Minted, Total Retired, Available, Price (USDC/tonne)
4. Buy Credits section – amount input, USDC cost calculator, "Buy Credits" button that calls `wagmi writeContract`
5. Provenance Timeline – showing: Sensor Reading → Oracle Verified → Token Minted → [Retired?] with timestamps
6. Retirement History table – from `RetirementLedger`, columns: Date, Buyer (ENS or truncated address), Amount, Reason

Use `wagmi v2` hooks: `useReadContract` for data, `useWriteContract` for transactions. Show loading states. Handle wallet not connected state. Make it mobile responsive.

Prompt 05 — Oracle Simulator

Claude Code

Write a Node.js script at `scripts/simulate-oracle.ts` that simulates IoT sensor data feeding into the Chainlink oracle system for local testing.

The script should:

1. Generate realistic CO2 sequestration readings (3000–8000 kg/day, seasonal variation)
2. Sign readings with a mock device private key using `ethers.js`
3. Upload signed JSON to IPFS via Pinata (use `PINATA_JWT` from `.env`)
4. Call `OracleConsumer.requestSequestration(projectId, ipfsCID)` on local Hardhat node
5. Wait for Chainlink oracle callback (mock using `hardhat-chainlink helper`)
6. Verify `CreditMinted` event was emitted with correct data
7. Log: CID, tokenId, amount minted, transaction hash

Mock sensor data format:

```
{ projectId, co2_kg, lat, lng, altitude, temperature, humidity, timestamp, deviceId, signature }
```

Run with: `npx ts-node scripts/simulate-oracle.ts --projectId 1 --days 7`

Also write a `scripts/mint-demo.ts` that mints 100 credits to a specified address for frontend demo purposes.

Prompt 06 — The Graph Subgraph

Claude Code

Create a The Graph subgraph for the CarbonToken protocol at `/subgraph`.

Files needed:

1. `subgraph.yaml` – index CarbonToken and RetirementLedger contracts on Base Sepolia
2. `schema.graphql` – define these entities:
 - `Project { id, name, location, totalMinted, totalRetired, credits[] }`
 - `CarbonCredit { id, tokenId, projectId, vintage, tonnesCO2, iotDataHash, mintedAt, retiredAt, retiree }`
 - `Retirement { id, tokenId, retiree, amount, reason, timestamp, transactionHash }`
 - `DailyStats { id, date, mintedAmount, retiredAmount, activeCredits }`
3. `src/mappings.ts` – handle events:
 - `handleCreditMinted` → create/update CarbonCredit and Project
 - `handleCreditRetired` → update CarbonCredit, create Retirement, update DailyStats

Include example GraphQL queries in `queries.graphql`:

- Get all active credits for a project
- Get retirement history for an address
- Get daily stats for charting
- Get total protocol stats (all-time minted, retired, active)

Deploy command: `graph deploy --studio carbon-credit-protocol`

Design a beautiful retirement certificate component for a carbon credit protocol.

Route: /certificate/[retirementId]

The certificate should look like a premium document:

- Dark header with company/protocol logo and "Carbon Retirement Certificate" title
- Certificate number (from RetirementLedger retirementId)
- Large central statement: "This certifies that [RETIREE_ENS_OR_ADDRESS] has permanently retired [AMOUNT] tonnes of CO₂ equivalent carbon credits"
- Project details: name, location, vintage year, IoT verification badge
- Metadata grid: retirement date, token ID, transaction hash (linked to block explorer), IPFS data hash
- Green "PERMANENTLY RETIRED" watermark/stamp effect
- "Verify on-chain" button that links to Etherscan
- Print/download as PDF button using browser print API

Fetch data from RetirementLedger.getRetirement(retirementId) using wagmi useReadContract. Include an animated checkmark on load using framer-motion.

Make it look professional enough that a Fortune 500 sustainability team would frame it.

Prompt 08 — Deployment Script

Claude Code

Write a complete Foundry deployment script at script/Deploy.s.sol for deploying the full carbon credit protocol to Base Sepolia.

Deployment order:

1. Deploy ProjectRegistry
2. Deploy RetirementLedger
3. Deploy CarbonToken(projectRegistryAddr, retirementLedgerAddr, baseURI)
4. Deploy OracleConsumer(carbonTokenAddr, chainlinkToken, chainlinkOracle, jobId)
5. Set oracle address in CarbonToken: carbonToken.setOracle(oracleConsumerAddr)
6. Register a demo project in ProjectRegistry
7. Call OracleConsumer.fundWithLink(1 LINK) for test oracle calls

After deployment, output a deployments/base-sepolia.json with all contract addresses.

Also write a verification script that calls forge verify-contract for each deployed contract.

Include a README section explaining how to:

- Get Base Sepolia ETH from faucet
- Get testnet LINK from Chainlink faucet
- Set up Chainlink Any API job
- Run the full deployment: `forge script script/Deploy.s.sol --rpc-url base_sepolia --broadcast --verify`

09 — FOR VIBE CODERS

Vibe Coder Tips

Speed-running a 36-hour hackathon build — shortcuts, gotchas, and sanity savers

01 Mock the Oracle First, Wire It Last

Don't spend hour 1 fighting Chainlink setup. Write a MockOracle.sol with a manual triggerMint() function you call from the frontend. Build the entire app against the mock. Wire the real Chainlink oracle in hour 30 when everything else works. The judges don't see your test code — they see your demo.

02 Use Foundry Anvil Fork for Local Dev

Run `anvil --fork-url https://base-sepolia.g.alchemy.com/v2/YOUR_KEY` to get a local fork of Base Sepolia with real contracts available. Deploy your contracts on top. You get real USDC, real Uniswap pools — and 10,000 ETH test accounts. No faucet grinding.

03 Use Scaffold-ETH 2 as Your Frontend Boilerplate

Don't scaffold from scratch. `npm create-eth@latest` gives you Next.js + wagmi + Hardhat + contract hot-reload in one command. Delete their example contracts, drop in yours, and you're building UI in 10 minutes instead of 2 hours of config hell.

04 Pin One Sample Dataset to IPFS on Day 1

Create one realistic mock IoT JSON with 30 days of sensor readings and pin it to Pinata before the hackathon starts. Use that CID everywhere. Your demo will never fail because of IPFS upload issues. Hard-code the CID in your mint script. You can swap real data in post-hackathon.

05 The Retirement Certificate IS the Demo

Spend 2 hours making the retirement certificate beautiful. It's the most photogenic moment: wallet connects → buys credit → retires it → certificate appears. If the certificate looks like a real document a sustainability officer would share on LinkedIn, you win the room. Use CSS print styles so it's downloadable as PDF instantly.

06 ERC-1155 tokenId Collision Gotcha

If you use `keccak256(projectId, vintage)` for tokenId generation, two projects with the same vintage will collide. Use `keccak256(abi.encodePacked(projectId, vintage, block.timestamp))` or a simple auto-incrementing counter with a mapping. Many teams lose hours to this. Test with two projects same vintage before the demo.

07 wagmi v2 Breaking Change: No More useContractWrite

If you copy examples from 2023, they'll use `useContractWrite` which doesn't exist in wagmi v2. The correct hook is `useWriteContract`. The callback pattern also changed — use `useWaitForTransactionReceipt({ hash })` separately. Use the official wagmi v2 docs, not Stack Overflow answers older than mid-2024.

08

Skip Uniswap Pool Creation, Use a Price Mock

Creating a real Uniswap v3 pool with concentrated liquidity takes 4+ hours to do correctly. For the hackathon, display a hardcoded price (\$18.50/tonne) with a realistic-looking chart. Put "Live pricing via Uniswap v3 pool — integration in progress" in small text. No judge will fault you. Use those 4 hours on the oracle or certificate instead.

09 Deploy to Base Sepolia, Not Localhost

Always demo on a public testnet, never localhost. "Let me just start my local node real quick" has ended more hackathon demos than anything else. Deploy contracts on Base Sepolia at hour 20. The 5-minute deploy pays dividends in demo reliability. Share the Basescan link with judges — it proves you shipped real code.

10 One-Line Slither Check Before Submitting

Run `slither contracts/ --print human-summary` in the last hour. Fix any HIGH severity findings (reentrancy, unchecked returns). Takes 10 minutes and makes your code review section of the demo much stronger. Judges with smart contract backgrounds will ask about security — have an answer ready.

10 — BUILD TIMELINE

36-Hour Hackathon Schedule

Hour-by-hour build plan for a team of 2–3

- H 0–2

●

Setup & Foundation
Scaffold project (Scaffold-ETH 2), set up .env, deploy MockOracle.sol to Anvil, create demo IPFS dataset
- H 2–8

●

Core Contracts
Write ProjectRegistry + CarbonToken + RetirementLedger. Write Foundry tests. Aim for 80%+ coverage.
- H 8–12

●

Oracle Integration
Write OracleConsumer.sol. Wire MockOracle for local. Write oracle simulator script. Test full mint flow end-to-end locally.
- H 12–20

●

Frontend Sprint
Build all 4 core pages: landing, project detail (with IoT chart), buy flow, retirement flow. Wire wagmi hooks. Ship retirement certificate page.
- H 20–24

●

Testnet Deploy
Deploy all contracts to Base Sepolia. Verify on Basescan. Point frontend to testnet. Run full E2E flow: register project → mint → buy → retire → certificate.
- H 24–28

●

The Graph + Real Chainlink
Deploy subgraph to The Graph Studio. Wire real Chainlink Any API if time permits. Otherwise keep mock — focus on polish.
- H 28–33

●

Polish & Buffer
UI polish, loading states, error handling, mobile responsiveness. Run Slither. Fix any HIGH findings. Buffer for unexpected issues.
- H 33–36

●

Demo Prep
Practice demo script 3 times. Fund demo wallets with testnet ETH. Seed 3 demo projects with pre-minted credits. Prepare slide with architecture diagram.

11 — RISK MATRIX

Risk & Mitigation

Risk	Likelihood	Severity	Mitigation
Chainlink oracle not working on testnet	High	High	Build MockOracle first. Wire real Chainlink only after everything else works. Demo uses mock — mention real oracle integration planned.
Uniswap v3 pool setup fails	High	Medium	Skip entirely. Display hardcoded price. Note in demo: "DEX integration ready for mainnet launch."
IPFS uploads too slow during demo	Medium	Medium	Pre-upload all demo data to Pinata before hackathon. Hard-code CIDs in demo scripts.
wagmi v2 API confusion	High	Low	Bookmark wagmi.sh v2 docs. Avoid Stack Overflow. Use the agent prompts in this doc which use correct v2 syntax.
ERC-1155 tokenId collision	Medium	High	Use auto-incrementing counter for tokenId. Test with identical projectId+vintage combination explicitly.
Testnet ETH/LINK insufficient	Low	Medium	Request from faucets 24h before hackathon. Keep 5 funded wallets. Have backup wallets with private keys ready.
Demo wallet has no credits to retire	Medium	High	Pre-mint 100 credits to 3 different demo wallets before demo. Test retirement flow 5 times before presenting.

12 — DEMO SCRIPT

The 3-Minute Demo Script

The exact sequence judges will remember. Practice it 5 times.

0:00

The Hook (30 seconds)

"89% of carbon credits sold today have never been independently verified. A \$420 billion market running on PDF certificates and broker spreadsheets. We replaced that with IoT sensors, a Chainlink oracle, and a burn function."

0:30

Show the Sequestration Chart (45 seconds)

Open /projects/1. Point to the live IoT chart: "This is real sensor data from a reforestation site in Tanzania. Every data point is signed by the device, verified by Chainlink, and the CID is stored on-chain. Click any point — here's the raw IPFS data."

1:15

Buy a Credit (30 seconds)

Connect wallet (MetaMask, pre-funded). "I'm buying 5 tonnes of CO₂ offsets." Click Buy. Transaction confirms. Show token in wallet. "That's an ERC-1155 token. Transferable, composable, and backed by a specific IoT reading you can verify."

1:45

Retire the Credit (45 seconds)

Navigate to /retire. Select the 5 credits. Enter reason: "Q4 2024 Scope 3 emissions offset." Click Retire. Transaction confirms. "Those tokens are now permanently burned. Irrevocably. And here's the retirement certificate."

2:30

The Close (30 seconds)

Show the certificate. "This certificate is verifiable by anyone in the world — the transaction hash links to Basescan, the IPFS hash links to the raw sensor data, and the retirement event is permanently on-chain. No broker. No PDF. No double-counting. Ever." Hand it to the judge.

Judge Questions to Prepare For

1. "How do you prevent a bad actor from spoofing IoT data?" → Device private keys registered in ProjectRegistry. Oracle verifies ECDSA signature against registered pubkey. Audit by third-party registry as backup layer.
2. "Why Base and not Ethereum mainnet?" → Gas costs. A \$20 carbon credit shouldn't cost \$15 in gas. Base gives us \$0.001 transactions with Ethereum security.
3. "What stops the same sensor from reporting twice?" → Oracle checks `credits[tokenId].oracleTimestamp == 0`. Once minted for a given projectId+vintage combination, re-minting reverts.

13 — POST-HACKATHON

Roadmap to Mainnet

Phase	Timeline	Milestones	Key Risk
V1 — Hackathon	Week 0	MockOracle, ERC-1155, Retirement, demo UI on Base Sepolia	Demo reliability
V2 — Protocol	Month 1–2	Real Chainlink integration, Uniswap v3 pool, The Graph subgraph, security audit	Oracle reliability
V3 — Market	Month 3–4	Multi-chain (Polygon, Arbitrum), institutional buyer onboarding, API for corporates	Regulatory clarity
V4 — Mainnet	Month 5–6	Base mainnet launch, 3 live IoT projects, integration with Verra/Gold Standard registry APIs	Smart contract bugs
V5 — Scale	Month 7–12	DAO governance for registry, fractional credit NFTs, carbon futures derivatives, mobile app	Execution bandwidth

The Long-Term Vision

Every tonne of carbon removed from the atmosphere, anywhere in the world, has an immutable on-chain record linking the physical sensor reading to the financial asset to the final retirement event. No intermediary can profit from double-counting. No audit report can be backdated. The planet gets a permanent, public ledger of climate action.